Elsevier DS Interview

Git & Python Code Quality

Detailed Q&A Revision Guide

Nityaa Kalra | August 2026

SECTION 1: GIT

Q1: What is the difference between git merge and git rebase?

Both integrate changes from one branch into another but in different ways.

git merge creates a new merge commit that ties together the histories of both branches. The original history is fully preserved — you can see exactly where branches diverged and rejoined. Use this when integrating a completed feature into main.

git rebase replays your commits on top of another branch as if you had started from there. The result is a clean linear history with no merge commit. Use this to clean up a feature branch before opening a PR.

```
# Merge — preserves full history git checkout main git merge feature/x # Rebase — rewrites history linearly git checkout feature/x git rebase main
```

CRITICAL: Never rebase a shared/public branch. It rewrites history and breaks everyone else's copy.

Q2: What happens during a merge conflict and how do you resolve it?

A conflict occurs when two branches modify the same part of the same file and Git cannot automatically decide which version to keep.

```
Git marks the conflict in the file with markers: <<<<<<< HEAD result = compute_macro_f1(predictions, labels) ======= result = compute_f1_score(preds, targets) >>>>>>> feature/new-metrics
```

Resolution steps:

1. Open the conflicted file and find the markers
2. Decide which version to keep — or combine them
3. Remove ALL conflict markers (<<<, ===, >>>)
4. git add <file>

5. git commit

To abort and start over: git merge --abort

Q3: What is the difference between git fetch and git pull?

git fetch downloads changes from the remote but does NOT integrate them into your working branch. Your local code is unchanged — you're just updating your local knowledge of what's on the remote. Safe to run at any time.

git pull = git fetch + git merge. Downloads and immediately merges into your current branch. Can cause conflicts if you have local uncommitted changes.

```
# Safe — just look at what changed git fetch origin git log HEAD..origin/main # see what's new # Download and merge immediately git pull origin main
```

Q4: What is git stash and when would you use it?

git stash temporarily shelves changes that aren't ready to commit, so you can switch context without losing your work.

Use case: you're halfway through a feature when someone asks you to urgently fix a bug on main. Stash your work, fix the bug, pop your stash and continue.

```
git stash # save current changes git checkout main # switch branch # fix the bug git checkout feature/x git stash pop # restore saved changes git stash list # see all stashes git stash drop # delete top stash git stash apply stash@{2} # apply specific stash
```

Q5: Walk me through a typical PR (Pull Request) workflow.

This is how professional teams work with Git. Know this cold.

```
1. git checkout -b feature/stance-evaluation # create feature branch 2. # write code, make commits 3. git add . && git commit -m 'Add class-level shift metric' 4. git push origin feature/stance-evaluation # push to remote 5. Open Pull Request on GitHub against main 6. Code review — teammates leave comments 7. Address comments, push more commits 8. PR approved — merge (or squash merge) 9. git branch -d feature/stance-evaluation # delete local branch
```

Squash merge: combines all your PR commits into one clean commit on main. Keeps main history readable.

Q6: What is .gitignore and what should you put in it for an ML project?

.gitignore tells Git which files/folders to never track. Essential for ML projects to avoid committing large files or secrets.

```
# Python __pycache__/ *.pyc .env # ML artifacts (too large for Git) models/ *.pt *.bin data/ *.csv # Jupyter .ipynb_checkpoints/ # Secrets .env config_secrets.json # OS .DS_Store
```

Use DVC (Data Version Control) for tracking large data and model files instead of Git.

Q7: What is git cherry-pick and when would you use it?

git cherry-pick applies a specific commit from one branch onto another, without merging the entire branch.

Use case: you fixed a critical bug on a feature branch and need that fix on main immediately, without merging all the unfinished feature work.

```
git log feature/x # find the commit hash git checkout main git cherry-pick abc1234 # apply just that commit
```

Q8: What is the difference between git reset --hard and git reset --soft?

git reset --soft HEAD~1: undoes the last commit but keeps the changes staged. You can recommit with a different message or after more edits.

git reset --hard HEAD~1: undoes the last commit AND discards all the changes. The work is gone. Use with caution.

git reset HEAD~1 (no flag = --mixed): undoes commit and unstages changes but keeps the files.

Memory trick: soft = keep staged, mixed = keep unstaged, hard = delete everything

SECTION 2: PYTHON CODE QUALITY

Q9: What is flake8 and what does it check?

flake8 is a lightweight Python linter that checks code style compliance against PEP8 — Python's official style guide. It flags issues but doesn't fix them.

What it catches:

- Lines longer than 79 characters (E501)

- Unused imports (F401)

- Undefined names (F821)

- Whitespace issues (E2xx)

- Missing blank lines between functions (E302)

```
# Run flake8 flake8 my_script.py flake8 . --max-line-length=100 # override line length flake8 . --ignore=E501,W503 # ignore specific rules
```

Q10: What is the difference between flake8, pylint, and black?

These are three different tools with different purposes — often used together:

Tool	Purpose	Fixes Code?	Speed
flake8	Style checker (PEP8)	No — flags only	Fast
pylint	Deep code analysis	No — flags only	Slow
black	Auto-formatter	Yes — rewrites	Fast

pylint catches deeper issues: unused variables, method arguments that don't match signatures, missing docstrings, code complexity scores. Much more opinionated.

black just fixes your code automatically — consistent quotes, line breaks, spacing. No configuration needed. Opinionated by design.

```
# Typical team setup flake8 . # check style pylint src/ # deep analysis black . # auto-format isort . # sort imports
```

Q11: What is isort and mypy?

isort automatically sorts your Python imports into a standard order: standard library first, then third-party, then local imports. Keeps import sections clean and consistent across the team.

mypy is a static type checker. It reads Python type annotations and checks for type errors before you run the code. Catches bugs like passing a string where an int is expected.

```
# isort example — before import torch import os import numpy as np from mymodule import helper # isort example — after import os import numpy as np import torch from mymodule import helper
```

```
# mypy type checking def compute_f1(predictions: list[int], labels: list[int]) -> float: ... compute_f1('wrong', [1, 2]) # mypy catches this error
```

Q12: How do linters fit into a CI/CD pipeline?

Linters run automatically on every push or PR via CI/CD. If linting fails, the PR is blocked — developers cannot merge until they fix the issues. This enforces consistent code quality across the entire team without relying on individuals to remember to run checks manually.

```
# .github/workflows/lint.yml on: [push, pull_request] jobs: lint: runs-on: ubuntu-latest steps: - uses: actions/checkout@v3 - run: pip install flake8 black pylint - run: black --check . # fails if formatting needed - run: flake8 . - run: pylint src/
```

Q13: What is a virtual environment and why use it?

A virtual environment is an isolated Python environment with its own packages and Python version, separate from your system Python. It prevents dependency conflicts between projects.

```
# Create and activate python -m venv venv source venv/bin/activate # Mac/Linux venv\Scripts\activate # Windows # Install dependencies pip install -r requirements.txt # Freeze current packages pip freeze > requirements.txt
```

Modern alternative: use uv (ultra-fast pip replacement) — you're already using it in your project: uv pip install -r requirements.txt

Q14: What makes Python code 'production-ready'? (JD specifically asks for this)

The JD mentions 'well-structured, production-ready Python modules' — here's what that means:

- **Modular**: functions do one thing, classes have clear responsibilities

- **Typed**: type annotations on function signatures

- **Documented**: docstrings on functions and classes

- **Tested**: unit tests for core logic

- **Logged**: uses logging module, not print statements

- **Config-driven**: parameters in config files, not hardcoded

- **Error handling**: try/except with meaningful error messages

- **Reproducible**: random seeds set, dependencies pinned

```
# Example: production-ready function import logging from typing import Optional logger = logging.getLogger(__name__) def compute_macro_f1( predictions: list[int], labels: list[int], num_classes: int = 3 ) -> Optional[float]: """Compute macro-averaged F1 across all classes. Args: predictions: Model output label indices labels: Ground truth label indices num_classes: Number of stance classes Returns: Macro F1 score or None if computation fails """ try: # computation logger.info(f'Computed F1 on {len(labels)} samples') return score except Exception as e: logger.error(f'F1 computation failed: {e}') return None
```

Q15: What is the difference between logging and print in Python?

print() is for development only. In production code you use the logging module because:

- Logging has severity levels: DEBUG, INFO, WARNING, ERROR, CRITICAL

- You can turn off debug logs in production without changing code

- Logs can be written to files, sent to monitoring systems

- Logs include timestamps, module names, line numbers automatically

```
import logging logging.basicConfig(level=logging.INFO) logger =
logging.getLogger(__name__) logger.debug('Detailed debug info') # dev only
logger.info('Model loaded successfully') # normal ops logger.warning('Low confidence
score') # something off logger.error('Inference failed') # something wrong
```

SECTION 3: QUICK REFERENCE CHEAT SHEET

GIT COMMANDS AT A GLANCE

Command	What it does
`git init`	Initialise new repo
`git clone <url>`	Copy remote repo locally
`git status`	Show changed files
`git add .`	Stage all changes
`git commit -m "msg"`	Commit staged changes
`git push origin main`	Push to remote
`git pull origin main`	Fetch + merge from remote
`git fetch`	Download remote changes (no merge)
`git branch -a`	List all branches
`git checkout -b feature/x`	Create and switch to branch
`git merge feature/x`	Merge branch into current
`git rebase main`	Replay commits on top of main
`git stash`	Shelve uncommitted changes
`git stash pop`	Restore shelved changes
`git log --oneline`	Compact commit history
`git diff HEAD`	Show unstaged changes
`git reset --soft HEAD~1`	Undo last commit, keep changes staged
`git reset --hard HEAD~1`	Undo last commit, delete changes
`git cherry-pick <hash>`	Apply specific commit

PYTHON CODE QUALITY TOOLS AT A GLANCE

Tool	Purpose	Run command
flake8	PEP8 style checker	`flake8 .`
black	Auto-formatter	`black .`
pylint	Deep code analysis	`pylint src/`
isort	Sort imports	`isort .`
mypy	Static type checking	`mypy src/`
pytest	Unit testing	`pytest tests/`
coverage	Test coverage	`coverage run -m pytest`
bandit	Security linting	`bandit -r src/`

KEY INTERVIEW TIPS FOR THIS SECTION

* If asked about Git in a team setting — always mention PRs, code review, and branch protection on main.

* If asked about code quality — mention that linters run in CI/CD, not just locally.

* Connect to your project: 'In my stance detection project I used a config-driven setup with config.json and a common interface for each model — that's the same modularity principle.'

* flake8 vs pylint: flake8 is fast and checks style, pylint is slower and checks logic. Use both.

* black is non-negotiable in most modern Python teams — just run it, don't debate formatting.